Computing Exact Aspect Graphs of Curved Objects: Algebraic Surfaces*

Jean Ponce[1], *Sylvain Petitjean*[1], *and David J. Kriegman*[2]

[1] Dept. of Computer Science, University of Illinois, Urbana, IL 61801, USA
[2] Dept. of Electrical Engineering, Yale University, New Haven, CT 06520, USA

Abstract. This paper presents an algorithm for computing the exact aspect graph of an opaque solid bounded by a smooth algebraic surface and observed under orthographic projection. The algorithm uses curve tracing, cell decomposition, and ray tracing to construct the regions of the view sphere delineated by visual events. It has been fully implemented, and examples are presented.

1 Introduction

The aspect graph [25] is a qualitative, viewer-centered representation that enumerates all possible appearances of an object: The range of all possible viewpoints is partitioned into maximal regions such that the structure of the image contours, also called the aspect, is the same from every viewpoint in a region. The change in the aspect at the boundary between regions is named a visual event. The maximal regions and their boundaries are organized into a graph, whose nodes represent the regions with their associated aspects and whose arcs correspond to the visual event boundaries between adjacent regions.

Since their introduction by Koenderink and Van Doorn [25] more than ten years ago, aspect graphs have been the object of very active research. The main focus has been on polyhedra, whose contour generators are viewpoint-independent. Indeed, approximate aspect graphs of polyhedra have been successfully used in recognition tasks [7, 18, 20], and several algorithms have been proposed for computing the exact aspect graph of these objects [6, 15, 16, 31, 36, 38, 39, 41, 42].

Recently, algorithms for constructing the exact aspect graph of simple curved objects such as solids bounded by quadric surfaces [8] and solids of revolution [11, 12, 26] have also been introduced. For more complex objects, it was recognized from the start that the necessary theoretical tools could be found in catastrophe theory [1, 5, 25]. However, algorithms based on these tools have, until very recently, remained elusive: Koenderink [24] and Kergosien [23] show the view sphere curves corresponding to the visual events of some surfaces, but, unfortunately, neither author details the algorithm used to compute these curves. Rieger [35] uses cylindrical algebraic decomposition to compute the aspect graph of a quartic surface of the form $z = f(x, y)$.

This paper is the third in a series on the construction of exact aspect graphs of smooth objects, based on the catalogue of possible visual events established by Kergosien [22] (see [33, 40] for the case of piecewise-smooth objects). Previously, we presented a fully implemented algorithm for solids of revolution whose generating curve is polynomial [26] (see [11, 12] for a different approach to the same problem), and reported preliminary results for polynomial parametric surfaces [32]. Here, we present a fully implemented algorithm for computing the aspect graph of an opaque solid bounded by parametric or

* This work was supported by the National Science Foundation under Grant IRI-9015749.

implicit smooth algebraic surfaces, observed under orthographic projection (see [23, 24, 35, 37] for related approaches).

This algorithm is described in Sect. 3. It relies on a combination of symbolic and numerical techniques, including curve tracing and cell decomposition [29], homotopy continuation [30], and "symbolic" ray tracing [21, 28]. An implementation is described in Sect. 4, and examples are presented (Figs. 4,5). Finally, future research directions are briefly discussed in Sect. 5. While the main ideas of our approach are presented in the body of the paper, detailed equations and algorithms are relegated to four appendices.

2 Visual Events

Let us start by reviewing some results from catastrophe theory [1]: From most viewpoints, the image contours of smooth surfaces are piecewise-smooth curves whose only singularities are cusps and t-junctions. The contour structure is in general stable with respect to viewpoint, i.e., it does not change when the camera position is submitted to a small perturbation. From some viewpoints, however, almost any perturbation of the viewpoint will alter the contour topology. A catalogue of these "visual events" has been established by Kergosien [22] for transparent generic smooth surfaces observed under orthographic projection (Fig. 1).

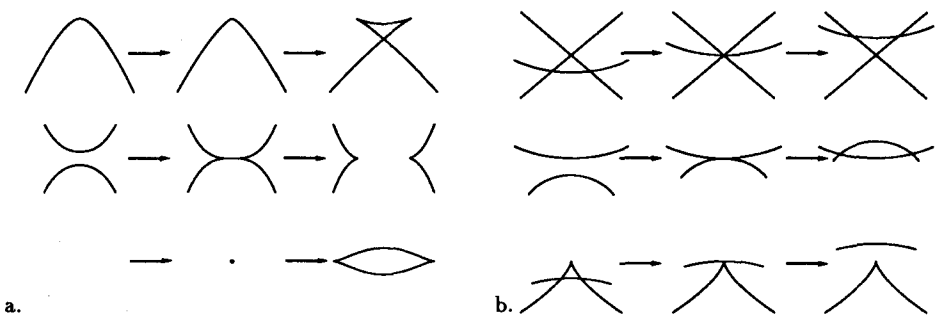

Fig. 1. Visual events. a. Local events. From top to bottom: swallowtail, beak-to-beak, lip. b. Multilocal events. From top to bottom: triple point, tangent crossing, and cusp crossing.

Each visual event in this catalogue occurs when the viewing direction has high order contact with the observed surface along certain characteristic curves [1, 24]. When contact occurs at a single point on the surface, the event is said to be local; when it occurs at multiple points, it is said to be multilocal. A catalogue of visual events is also available for piecewise-smooth surfaces [33, 40], but we will restrict our discussion to smooth surfaces in the rest of this paper.

2.1 Local Events

As shown in [22], smooth surfaces may exhibit three types of local events: swallowtail, beak-to-beak, and lip transitions (Fig. 1.a). During a swallowtail transition, a smooth image contour forms a singularity and then breaks off into two cusps and a t-junction. In a beak-to-beak transition, two distinct portions of the occluding contour meet at a point

in the image. After meeting, the contour splits and forms two cusps; the connectivity of the contour changes. Finally, a lip transition occurs when, out of nowhere, a closed contour is formed with the introduction of two cusps.

Swallowtails occur on flecnodal curves, and both beak-to-beak and lip transitions occur on parabolic curves [1, 24]. Flecnodal points are inflections of asymptotic curves, while parabolic points are zeros of the Gaussian curvature. Equations for the parabolic and flecnodal curves of parametric and implicit surfaces are given in Appendices A.1 and B.1 respectively. The corresponding viewing directions are asymptotic directions along these curves.

2.2 Multilocal Events

These events occur when two or more surface points project onto the same contour point. As shown in [22], there are three types of multilocal events: triple points, tangent crossings, and cusp crossings (Fig. 1.b). A triple point is formed by the intersection of three contour segments. For an opaque object, only two branches are visible on one side of the transition while three branches are visible on the other side. A tangent crossing occurs when two contours meet at a point and share a common tangent. Finally, a cusp crossing occurs when the projection of a contour cusp meets another contour.

A multilocal event is characterized by a curve defined in a high dimension space, or equivalently by a family of surface curves. For example, a triple point is formed when three surface points are aligned and, in addition, the surface normals at the three points are all orthogonal to the common line supporting these points. By sweeping this line while maintaining three-point contact, a family of three curves is drawn on the surface. Equations for the families of surface curves corresponding to multilocal events are given in Appendices A.2 and B.2 for parametric and implicit surfaces respectively. The corresponding viewing directions are parallel to the lines supporting the points forming the events.

3 The Algorithm

We propose the following algorithm for constructing the aspect graph of an opaque solid bounded by an algebraic surface:

1. Trace the visual event curves.
2. Eliminate the occluded events.
3. Construct the regions delineated on the view sphere by the remaining events.
4. Construct the corresponding aspects.

We now detail each step of the algorithm. Note that the aspect graph of a transparent solid can be constructed by using the same procedure but omitting step 2.

3.1 Step 1: Tracing Visual Events

As shown in Sect. 2, a visual event corresponds in fact to two curves: a curve (or family of curves) Γ drawn on the object surface and a curve Δ drawn on the view sphere.

For algebraic surfaces, the curve Γ is defined implicitly in $\mathbb{R}^{n+1}$ by a system of n polynomial equations in $n+1$ unknowns, with $1 \leq n \leq 8$ (see Appendices A and B):

$$\begin{cases} P_1(X_0, X_1, \ldots, X_n) = 0, \\ \ldots \\ P_n(X_0, X_1, \ldots, X_n) = 0. \end{cases} \tag{1}$$

To trace a visual event, we first trace Γ in $\mathbb{R}^{n+1}$. We then trace Δ by mapping points of Γ onto points of Δ: given a point on Γ, the corresponding point on Δ is an asymptotic direction for local events, or the direction of the line joining two surface points for multilocal events.

The curve tracing algorithm is decomposed into the following steps (Fig. 2): 1.1. Compute all extremal points of Γ in some direction, say X_0 (this includes all singular points). 1.2. Compute all intersections of Γ with the hyperplanes orthogonal to the X_0 axis at the extremal points. 1.3. For each interval of the X_0 axis delimited by these hyperplanes, intersect Γ and the hyperplane passing through the mid-point of the interval to obtain one sample for each real branch. 1.4. March numerically from the sample points found in step 1.3 to the intersection points found in step 1.2 by predicting new points through Taylor expansion and correcting them through Newton iterations.

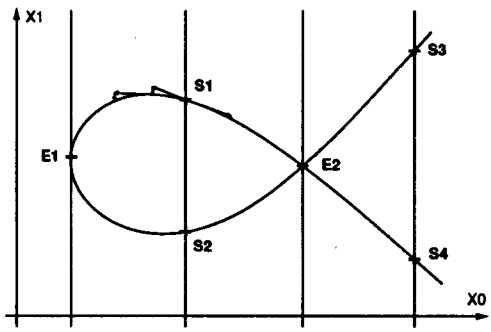

Fig. 2. An example of curve tracing in $\mathbb{R}^2$. This curve has two extremal points $\mathbf{E}_1, \mathbf{E}_2$, and four regular branches with sample points $\mathbf{S}_1$ to $\mathbf{S}_4$; note that $\mathbf{E}_2$ is singular.

This algorithm overcomes the main difficulties of curve tracing, namely finding a sample point on every real branch and marching through singularities. Its output is a graph whose nodes are extremal or singular points on Γ and whose arcs are discrete approximations of the smooth curve branches between these points. This graph is similar to the s-graph representation of plane curves constructed through cylindrical algebraic decomposition [2]. Using the mapping from Γ onto Δ, a discrete approximation of the curve Δ is readily constructed.

Technical Details. We now detail the computations involved in the curve tracing algorithm. The casual reader may want to skip the rest of this section, at least on first reading, and jump ahead to Sect. 3.2.

Step 1.1 requires the computation of the extrema of Γ in the X_0 direction. As shown in Appendix C, these points are the solutions of the system of $n+1$ polynomial equations in $n+1$ unknowns obtained by adding the equation $|J| = 0$ to system (1). Here, J is the Jacobian matrix $(\partial P_i/\partial X_j)$, with $i, j = 1, .., n$. Steps 1.2 and 1.3 require computing the intersections of a curve with a hyperplane, and these points are once again the solutions of a system of polynomial equations. We use the homotopy continuation method, as described in Appendix D, to solve these equations.

The curve is actually traced in step 1.4, using a classical prediction/correction approach based on a Taylor expansion of the P_i's [4, 13]. As shown in Appendix C, this

involves inverting the matrix J which is guaranteed to be nonsingular on extrema-free intervals. Note that all real branches can be traced in parallel. As shown in Appendix D, finding the extrema of the curve and its intersections with a family of hyperplanes is a parallel process too.

There is no conceptual difficulty in applying this algorithm to aspect graph construction, but there is a very practical problem: the visual events are defined by very high degree algebraic curves, and tracing them requires solving systems of equations that may have millions of roots. We will come back to this problem in Sect. 4.

3.2 Step 2: Eliminating Occluded Events

All visual events of the transparent object are found in step 1 of the algorithm. For an opaque object, some of these events will be occluded, and they should be eliminated. The visibility of an event curve Γ can be determined through ray tracing at its sample point found in step 1.3 [21, 44].

3.3 Step 3: Constructing the Regions

To construct the aspect graph regions delineated by the curves Δ on the view sphere, we refine the curve tracing algorithm into a cell decomposition algorithm whose output is a description of the regions, their boundary curves, and their adjacency relationships. Note that this refinement is only possible for curves drawn in two-dimensional spaces such as the sphere.

The algorithm is divided into the following steps (Fig. 3): 3.1. Compute all extremal points of the curves in the X_0 direction. 3.2. Compute all the intersection points between the curves. 3.3. Compute all intersections of the curves with the "vertical" lines orthogonal to the X_0 axis at the extremal and intersection points. 3.4. For each interval of the X_0 axis delimited by these lines, do the following: 3.4.1. Intersect the curves and the line passing through the mid-point of the interval to obtain a sample point on each real branch of each curve. 3.4.2. Sort the sample points in increasing X_1 order. 3.4.3. March from the sample points to the intersection points found in step 3.3. 3.4.4. Two consecutive branches within an interval of X_0 and the vertical segments joining their extremities bound a region.

A sample point can be found for each region as the mid-point of the sample points of the bounding curve branches. This point is used to construct a representative aspect in Sect. 3.4. Maximal regions are found by merging all regions adjacent along a vertical line segment (two regions are adjacent if they share a common boundary, i.e., a vertical line segment or a curve branch).

Technical Details. We now detail the computations involved in the cell decomposition algorithm. Again, the casual reader may want to skip the rest of this section, and jump ahead to Sect. 3.4.

In our application, the coordinates X_0, X_1 define a parameterization of the view sphere, such as spherical angles. Also, the curve Δ corresponding to a visual event is not explicitly defined by polynomial equations. As shown in Appendices A and B, it is actually possible to augment the polynomial equations defining Γ to construct a new algebraic curve Ω defined in $\mathbb{R}^{m+1}$, with $m > n$, such that Δ is the projection of Ω onto $\mathbb{R}^2$.

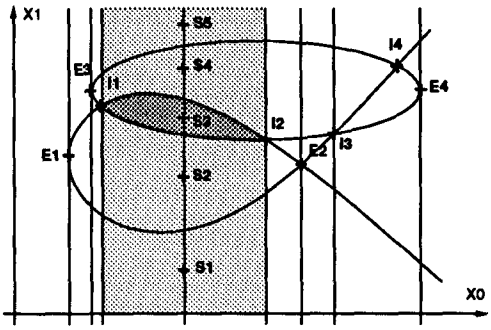

Fig. 3. An example of cell decomposition. Two curves are shown, with their extremal points $\mathbf{E}_i$ and their intersection points $\mathbf{I}_j$; the shaded rectangle delimited by $\mathbf{I}_1$ and $\mathbf{I}_2$ is divided into five regions with sample points $\mathbf{S}_1$ to $\mathbf{S}_5$; the region corresponding to $\mathbf{S}_3$ is shown in a darker shade.

The extrema of Δ in the X_0 direction are the projections of the extrema of Ω in this direction, and they can be found by solving a system of $m+1$ equations in $m+1$ unknowns through continuation. Similarly, the intersections of two curves Δ_1 and Δ_2 can be found by writing that Ω_1 and Ω_2 must project onto the same point in $\mathbb{R}^2$ and by solving the corresponding system of $2m$ equations in $2m$ unknowns. Marching along Δ is achieved by marching along Ω and projection onto the $\mathbb{R}^2$ plane.

An alternative is to first localize candidate regions of the view sphere where extrema and intersections may occur (using, for example, an adaptive subdivision of the sphere) and to converge to the actual points through local numerical optimization using the equations defining the visual events. This method does not involve the costly resolution of large sets of polynomial equations. A simpler version of this method is to directly work with the discrete approximations of the curves Δ obtained in step 1. This is the method we have actually used in the implementation described in Sect. 4.

3.4 Step 4: Constructing the Aspects

This step involves determining the contour structure of a single view for each region, first for the transparent object, then for the opaque object. This can be done through "symbolic" ray tracing of the object contour [28] as seen from the sample point of the region. Briefly, the contour structure is found using the curve tracing algorithm described earlier. Since contour visibility only changes at the contour singularities found by the algorithm, it is determined through ray tracing [21, 44] at one sample point per regular branch.

4 Implementation and Results

The algorithm described in Sect. 3 has been fully implemented. Tracing the visual event curves (step 1) is by far the most expensive part of the algorithm. Curve tracing and continuation are parallel processes that can be mapped onto medium-grained MIMD architectures. We have implemented continuation on networks of Sun SPARC Stations communicating via Ethernet, networks of INMOS Transputers, and Intel Hypercubes. In practice, this allows us to routinely solve systems with a few thousands of roots, a task

that requires a few hours using a dozen of Sparc Stations. The elimination of occluded events in step 2 of the algorithm only requires ray tracing a small number of points on the surface and takes a negligible amount of time. In our current implementation, the cell decomposition algorithm of step 3 works directly with discrete approximations of the visual event curves and only takes a few seconds. Finally it takes a few minutes to generate the aspects in step 4.

In the following examples, an object is represented graphically by its silhouette, parabolic and flecnodal curves, and the corresponding aspect graph is shown (more precisely, the visual event curves and their intersections are drawn on the view sphere).

Figure 4.a shows an object bounded by a complex parametric surface, and its (partial) aspect graph. All visual events except certain cusp crossings and triple points have been traced. Note that the aspect graph is extremely complicated, even though some events are still missing. Also, note that this object is in fact only piecewise-smooth. As remarked earlier, the catalogue of visual events used in this paper can be extended to piecewise-smooth surfaces [33, 40], and corresponding equations can also be derived [32].

The objects considered in the next three examples are described by smooth compact implicit surfaces of degree 4. The full aspect graph has been computed. Note that it has the structure predicted in [5, 24] for similarly shaped surfaces.

Figure 4.b shows the silhouette of a bean-shaped object and the corresponding aspect graph, with vertices drawn as small circles. This object has a hyperbolic patch within a larger convex region.

Figure 4.c shows a squash-shaped object, its parabolic and flecnodal curves and its aspect graph. This object has two convex parts separated by a hyperbolic region. Note the two concentric parabolic curves surrounding the flecnodal curves.

Figure 4.d shows a "dimpled" object and its aspect graph. This object has a concave island within a hyperbolic annulus, itself surrounded by a convex region. The flecnodal curve almost coincides with the outer parabolic curve (compare to [24, p. 467]). There is no tangent crossing in this case. Figure 5.a shows the corresponding decomposition of the parameter space of the view sphere into 16 maximal regions, with sample points indicated as black spots. The horizontal axis represents longitude, measured between $-\pi$ and π, and the vertical axis represents latitude, measured between $-\pi/2$ and $\pi/2$. Figure 5.b shows the corresponding 16 aspects.

What do these results indicate? First, it seems that computing exact aspect graphs of surfaces of high degree is impractical. It can be shown that triple points occur only for surfaces of degree 6 or more, and that computing the extremal points of the corresponding curves requires solving a polynomial system of degree 4,315,680 –a very high degree indeed! Even if this extraordinary computation were feasible (or another method than ours proved simpler), it is not clear how useful a data structure as complicated as the aspect graph of Fig. 4.a would be for vision applications.

On the other hand, aspect graphs of low-degree surfaces do not require tracing triple points, and the necessary amount of computation remains reasonable (for example, a mere few thousands of roots had to be computed for the tangent crossings of the bean-shaped object). In addition, as demonstrated by Fig. 4.b-d, the aspect graphs of these objects are quite simple and should prove useful in recognition tasks.

5 Discussion and Future Research

We have presented a new algorithm for computing the exact aspect graph of curved objects and described its implementation. This algorithm is quite general: as noted in [27],

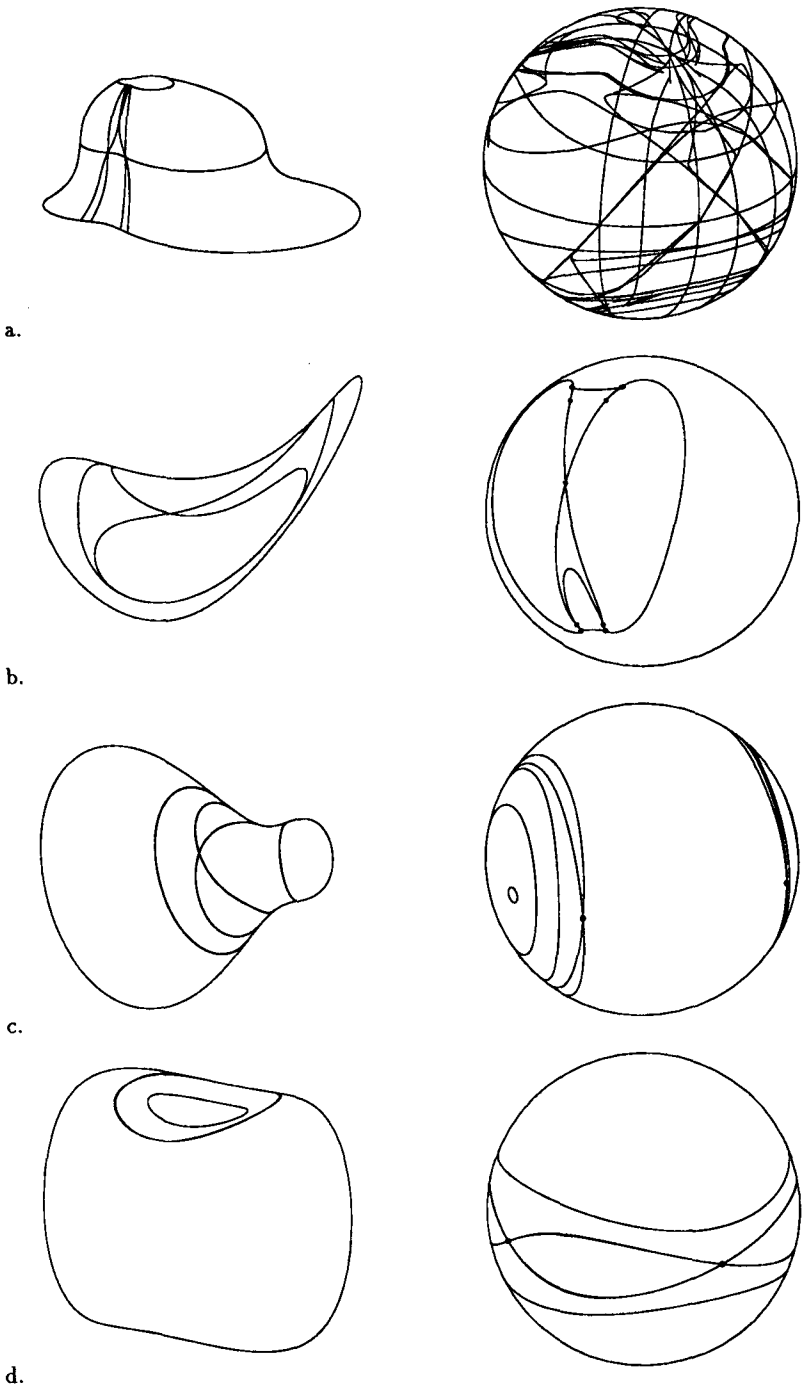

Fig. 4. A few objects and their aspect graphs: a. A parametric surface. b. A bean-shaped implicit surface. c. A squash-shaped implicit surface. d. A "dimpled" implicit surface.

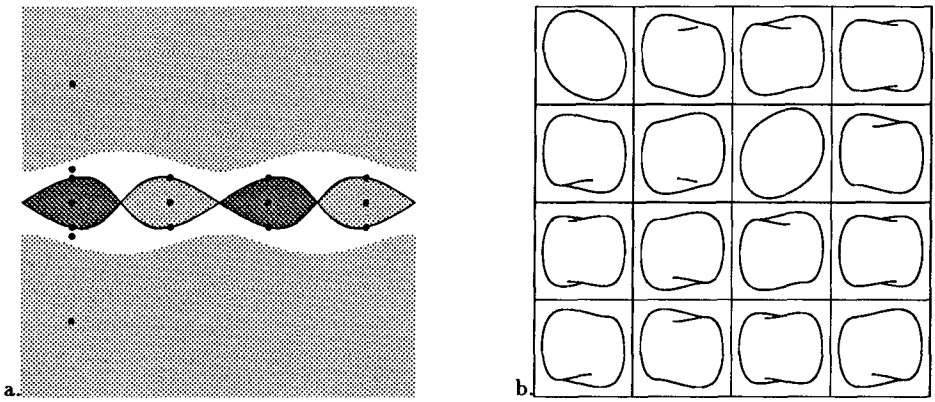

Fig. 5. a. Aspect graph regions of the "dimpled" object in parameter space. b. The corresponding aspects.

algebraic surfaces subsume most representations used in computer aided design and computer vision. Unlike alternative approaches based on cylindrical algebraic decomposition [3, 9], our algorithm is also practical, as demonstrated by our implementation.

We are investigating the case of perspective projection: the (families of) surface curves that delineate the visual events under orthographic projection also delineate perspective projection visual events by defining ruled surfaces that partition the three-dimensional view space into volumetric cells.

Future research will be dedicated to actually using the aspect graph representation in recognition tasks. In [27], we have demonstrated the recovery of the position and orientation of curved three-dimensional objects from monocular contours by using a purely quantitative process that fits an object-centered representation to image contours. What is missing is a control structure for guiding this process. We believe that the qualitative, viewer-centered aspect graph representation can be used to guide the search for matching image and model features and yield efficient control structures analogous to the interpretation trees used in the polyhedral world [14, 17, 19].

Acknowledgments: We thank Seth Hutchinson, Alison Noble and Brigitte Ponce for useful discussions and comments.

Appendix A: The Visual Events of Parametric Surfaces

A parametric algebraic surface is represented by:

$$\mathbf{X}(u,v) = (X(u,v), Y(u,v), Z(u,v))^T, \quad (u,v) \in I \times J \subset \mathbb{R}^2, \tag{2}$$

where X, Y, Z are (rational) polynomials in u, v. These surfaces include Bézier patches and non-uniform rational B-splines (NURBS) for example.

In this appendix and the following one, we assume that the viewing direction $\mathbf{V}$ is parameterized in $\mathbb{R}^2$, by spherical angles for example. Note that all equations involving $\mathbf{V}$ can be made polynomial by using the rational parameterization of the trigonometric functions.

A.1 Local Events

We recall the equations defining the surface curves (parabolic and flecnodal curves) associated to the visual events (beaks, lips, swallowtails) of parametric surfaces. There is nothing new here, but equations for flecnodal curves are not so easy to find in the literature.

Note: in this appendix, a u (resp. v) subscript is used to denote a partial derivative with respect to u (resp. v).

Consider a parametric surface $\mathbf{X}(u, v)$ and define:

$$\mathbf{N} = \mathbf{X}_u \times \mathbf{X}_v, \quad e = (\mathbf{X}_{uu} \cdot \mathbf{N})/|\mathbf{N}|, \quad f = (\mathbf{X}_{uv} \cdot \mathbf{N})/|\mathbf{N}|, \quad g = (\mathbf{X}_{vv} \cdot \mathbf{N})/|\mathbf{N}|, \quad (3)$$

i.e., $\mathbf{N}$ is the surface normal, and e, f, g, are the coefficients of the second fundamental form in the coordinate system $(\mathbf{X}_u, \mathbf{X}_v)$ [10].

A.1.1 Asymptotic Directions. The asymptotic curves are the surface curves $\mathbf{A}(t) = \mathbf{X}(u(t), v(t))$ defined by the differential equation:

$$eu'^2 + 2fu'v' + gv'^2 = 0. \quad (4)$$

The asymptotic directions are given by $u'\mathbf{X}_u + v'\mathbf{X}_v$, where u' and v' are solutions of the above equation. A contour cusp occurs when the viewing direction is an asymptotic direction.

A.1.2 Parabolic Curves. The parabolic curves of a parametric surface $\mathbf{X}(u, v)$ are given by:

$$eg - f^2 = 0. \quad (5)$$

For each point $\mathbf{X}(u, v)$ along a parabolic curve, there is only one asymptotic direction, which is given by (4). In the language of Sect. 3, (4) defines the mapping from Γ (the parabolic curve) onto Δ (the view sphere curve corresponding to beak-to-beak and lip events). Equivalently, Δ is the projection of the curve Ω obtained by adding to (5) the equations $\mathbf{V} \times (u'\mathbf{X}_u + v'\mathbf{X}_v) = 0$ and (4).

A.1.3 Flecnodal Curves. As shown in [43, p.85], the flecnodal points are inflections of the asymptotic curves A, given by:

$$(\mathbf{A}' \times \mathbf{A}'') \cdot \mathbf{N} = 0, \quad (6)$$

which can be seen as an equation in u, v, t, or, equivalently, as an equation in u, v, u', v', u'', v''.

An equation for the flecnodal curves is obtained by eliminating u', v', u'', v'' among eqs. (4), (6), and the equation obtained by differentiating (4) with respect to t. Note that since all three equations are homogeneous in u', v' and in u'', v'', this can be done by arbitrarily setting $u' = 1, u'' = 1$, say, and eliminating v', v'' among these three equations. The resulting equation in u, v characterizes the flecnodal curves.

Note that although it is possible to construct the general equation of flecnodal curves for arbitrary parametric surfaces, this equation is very complicated, and it is better to derive it for each particular surface using a computer algebra system. As before, explicit equations for Ω can be constructed.

A.2 Multilocal Events

Multilocal events occur when the viewing direction $\mathbf{V}$ has high order contact with the surface in at least two distinct points $\mathbf{X}_1$ and $\mathbf{X}_2$. In that case, $\mathbf{V} = \mathbf{X}_1 - \mathbf{X}_2$.

A.2.1 Triple Points. The triple point is conceptually the simplest of the multilocal events. It occurs when three contour fragments intersect at a single point. Let $\mathbf{X}_i = \mathbf{X}(u_i, v_i)$, for $i = 1, 2, 3$, be the three corresponding surface points, and let $\mathbf{N}_i$ be the corresponding surface normals, we obtain the following equations:

$$\begin{cases} (\mathbf{X}_1 - \mathbf{X}_2) \times (\mathbf{X}_2 - \mathbf{X}_3) = 0, \\ (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_3 = 0, \\ (\mathbf{X}_2 - \mathbf{X}_3) \cdot \mathbf{N}_1 = 0, \\ (\mathbf{X}_3 - \mathbf{X}_1) \cdot \mathbf{N}_2 = 0. \end{cases} \tag{7}$$

The first equation is a vector equation (or equivalently a set of two independent scalar equations) that expresses the fact that the three points are aligned with the viewing direction. The next three equations simply express the fact that the three points belong to the occluding contour. It follows that triple points are characterized by five equations in the six variables $u_i, v_i, i = 1, 2, 3$.

An explicit equation for the curve Ω corresponding to a triple point can be obtained by replacing $(\mathbf{X}_2 - \mathbf{X}_3)$ by $\mathbf{V}$ in (7). Similar comments apply to the other multilocal events.

A.2.2 Tangent Crossings. A tangent crossing occurs when two occluding contour points $\mathbf{X}_1 = \mathbf{X}(u_1, v_1)$ and $\mathbf{X}_2 = \mathbf{X}(u_2, v_2)$ project to the same image point and have collinear surface normals $\mathbf{N}_1$ and $\mathbf{N}_2$. This can be rewritten as:

$$\begin{cases} \mathbf{N}_1 \times \mathbf{N}_2 = \mathbf{0}, \\ (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_1 = 0. \end{cases} \tag{8}$$

Again, remark that the first equation is a vector equation (or equivalently a set of two independent scalar equations). It follows that tangent crossings are characterized by three equations in the four variables $u_i, v_i, i = 1, 2$.

A.2.3 Cusp Crossings. A cusp crossing occurs when two occluding contour points $\mathbf{X}_1$ and $\mathbf{X}_2$ project to the same image point and one of the points, say $\mathbf{X}_1$, is a cusp. This can be rewritten as:

$$\begin{cases} (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_1 = 0, \\ (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_2 = 0, \\ e_1 a^2 + 2 f_1 ab + g_1 b^2 = 0, \end{cases} \tag{9}$$

where e_1, f_1, g_1 are the values of the coefficients of the second fundamental form at $\mathbf{X}_1$, and (a, b) are the coordinates of the viewing direction $\mathbf{X_1} - \mathbf{X}_2$ in the basis $\mathbf{X}_u(u_1, v_1)$, $\mathbf{X}_v(u_1, v_1)$ of the tangent plane. Note that a, b can be computed from the dot products of $\mathbf{X_1} - \mathbf{X}_2$ with $\mathbf{X}_u(u_1, v_1)$ and $\mathbf{X}_v(u_1, v_1)$. It follows that cusp crossings are characterized by three equations in the four variables $u_i, v_i, i = 1, 2$.

Appendix B: The Visual Events of Implicit Surfaces

An implicit algebraic surface is represented by:

$$F(X,Y,Z) = F(\mathbf{X}) = 0, \tag{10}$$

where F is a polynomial in X, Y, Z.

B.1 Local Events

For implicit surfaces, even the equations of parabolic curves are buried in the literature. Equations for both parabolic and flecnodal curves are derived in this appendix.

Note: in this appendix, X, Y, Z subscripts denote partial derivatives with respect to these variables.

B.1.1 Asymptotic Directions. An asymptotic direction $\mathbf{V}$ at a point $\mathbf{X}$ lies in the tangent plane and has second order contact with the surface. It is characterized by:

$$\begin{cases} \nabla F(\mathbf{X}) \cdot \mathbf{V} = 0, \\ \mathbf{V}^T H(\mathbf{X}) \mathbf{V} = 0, \end{cases} \tag{11}$$

where $H(\mathbf{X})$ is the Hessian of F at $\mathbf{X}$. Asymptotic directions are determined by solving this homogeneous system in $\mathbf{V}$.

B.1.2 Parabolic Curves. The parabolic curves of an implicit surface $F(\mathbf{X}) = 0$ are given by:

$$\begin{aligned} &F_X^2(F_{YY}F_{ZZ} - F_{YZ}^2) + F_Y^2(F_{XX}F_{ZZ} - F_{XZ}^2) \\ &\quad + F_Z^2(F_{XX}F_{YY} - F_{XY}^2) + 2F_XF_Y(F_{XZ}F_{YZ} - F_{ZZ}F_{XY}) \\ &\quad + 2F_YF_Z(F_{XY}F_{XZ} - F_{XX}F_{YZ}) + 2F_XF_Z(F_{XY}F_{YZ} - F_{YY}F_{XZ}) = 0, \end{aligned} \tag{12}$$

plus the equation $F(\mathbf{X}) = 0$ itself. For each point $\mathbf{X}$ along a parabolic curve, there is only one asymptotic direction, which is given by (11). It should be noted that one can directly characterize the beak-to-beak and lip events by adding to (12) the equations $F(\mathbf{X}) = 0$ and (11), and tracing the resulting curve Ω in $\mathbb{R}^5$; the projection of this curve onto $\mathbb{R}^2$ defines the beak-to-beak and lip curves on the view sphere.

B.1.3 Flecnodal Curves. A surface point $\mathbf{X} = (X, Y, Z)^T$ on a flecnodal curve has third order contact with a line along an asymptotic direction $\mathbf{V} = (V_1, V_2, V_3)^T$ [1]. This is characterized by:

$$\begin{cases} \nabla F(\mathbf{X}) \cdot \mathbf{V} = 0, \\ \mathbf{V}^T H(\mathbf{X}) \mathbf{V} = 0, \\ \mathbf{V}^T (H_X(\mathbf{X})V_1 + H_Y(\mathbf{X})V_2 + H_Z(\mathbf{X})V_3)\mathbf{V} = 0. \end{cases} \tag{13}$$

Since these three equations are homogeneous in the coordinates of $\mathbf{V}$, these coordinates can easily be eliminated to obtain a single equation in $\mathbf{X}$. Along with $F(\mathbf{X}) = 0$, this system defines the flecnodal curves. As before, explicit equations for Ω can be constructed.

B.2 Multilocal Events

B.2.1 Triple Points. Let $\mathbf{X}_i, i = 1,2,3$, be three points forming a triple point event. The corresponding equations are similar to the equations defining triple points of parametric surfaces:

$$\begin{cases} F(\mathbf{X}_i) = 0, \quad i = 1,2,3, \\ (\mathbf{X}_1 - \mathbf{X}_2) \times (\mathbf{X}_2 - \mathbf{X}_3) = 0, \\ (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_3 = 0, \\ (\mathbf{X}_2 - \mathbf{X}_3) \cdot \mathbf{N}_1 = 0, \\ (\mathbf{X}_3 - \mathbf{X}_1) \cdot \mathbf{N}_2 = 0, \end{cases} \tag{14}$$

where $\mathbf{N}_i = \nabla F(\mathbf{X}_i)$. It follows that triple points are characterized by eight equations in the nine variables $X_i, Y_i, Z_i, i = 1,2,3$.

B.2.2 Tangent Crossings. Again, the equations defining tangent crossings of implicit surfaces are similar to the corresponding equations for parametric surfaces:

$$\begin{cases} F(\mathbf{X}_i) = 0, \quad i = 1,2, \\ \mathbf{N}_1 \times \mathbf{N}_2 = \mathbf{0}, \\ (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_1 = 0. \end{cases} \tag{15}$$

This is a system of five equations in the six variables $X_i, Y_i, Z_i, i = 1,2$.

B.2.3 Cusp Crossings. Cusp crossings are characterized by:

$$\begin{cases} F(\mathbf{X}_i) = 0, \quad i = 1,2, \\ (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_1 = 0, \\ (\mathbf{X}_1 - \mathbf{X}_2) \cdot \mathbf{N}_2 = 0, \\ (\mathbf{X}_1 - \mathbf{X}_2)^T H(\mathbf{X}_1)(\mathbf{X}_1 - \mathbf{X}_2) = 0, \end{cases} \tag{16}$$

where the last equation simply expresses the fact that the viewing direction is an asymptotic direction of the surface at $\mathbf{X}_1$. This is again a system of five equations in the six variables $X_i, Y_i, Z_i, i = 1,2$.

Appendix C: Details of the Curve Tracing Algorithm

C.1 Step 1.1: Finding the Extremal Points

The extrema of Γ in the X_0 direction are given by differentiating (1) and setting $dX_0 = 0$:

$$\begin{cases} (\partial P_1/\partial X_1)dX_1 + \ldots + (\partial P_1/\partial X_n)dX_n = 0 \\ \ldots \\ (\partial P_n/\partial X_1)dX_1 + \ldots + (\partial P_n/\partial X_n)dX_n = 0 \end{cases} \Longleftrightarrow J \begin{pmatrix} dX_1 \\ \ldots \\ dX_n \end{pmatrix} = 0 \tag{17}$$

where $J = (\partial P_i/\partial X_j)$, with $i, j = 1, .., n$, is the Jacobian matrix. This system has nontrivial solutions if and only if the determinant $D(X_0, X_1, ..., X_n) = |J|$ of the Jacobian matrix vanishes. The extrema of Γ are therefore the solutions of:

$$\begin{cases} P_1(X_0, X_1, ..., X_n) = 0, \\ \ldots \\ P_n(X_0, X_1, ..., X_n) = 0, \\ D(X_0, X_1, ..., X_n) = 0. \end{cases} \tag{18}$$

C.2 Steps 1.2 and 1.3: Intersecting the Curve with Hyperplanes

These steps correspond to finding all the intersections of Γ with some hyperplane $X_0 = \hat{X}_0$. These intersections are given by:

$$\begin{cases} P_1(\hat{X}_0, X_1, ..., X_n) = 0, \\ \dots \\ P_n(\hat{X}_0, X_1, ..., X_n) = 0. \end{cases} \tag{19}$$

C.3 Step 1.4: Marching on Extrema-Free Intervals

To trace a curve on an extrema-free interval, we use a classical prediction/correction approach based on a first order Taylor expansion of the P_i's (higher order expansions could also be used [4, 13]). By differentiating (1), we obtain:

$$J \begin{pmatrix} dX_1 \\ \dots \\ dX_n \end{pmatrix} = -dX_0 \begin{pmatrix} \partial P_1/\partial X_0 \\ \dots \\ \partial P_n/\partial X_0 \end{pmatrix} \tag{20}$$

Given a step dX_0 in the X_0 direction, one can predict the remaining dX_i's by solving this system of linear equations. This is only possible when the determinant of the Jacobian matrix J is non-zero, which is exactly equivalent to saying that the point $(X_0, \dots, X_n)^T$ is not an extremum in the X_0 direction.

The correction step uses Newton iterations to converge back to the curve from the predicted point. We write once more a first order Taylor approximation of the P_i's to compute the necessary correction $(dX_1, \dots, dX_n)^T$ for a fixed value of X_0:

$$\begin{cases} P_1 + (\partial P_1/\partial X_1)dX_1 + \dots + (\partial P_1/\partial X_n)dX_n = 0 \\ \dots \\ P_n + (\partial P_n/\partial X_1)dX_1 + \dots + (\partial P_n/\partial X_n)dX_n = 0 \end{cases} \Longleftrightarrow \begin{pmatrix} dX_1 \\ dX_2 \\ \dots \\ dX_n \end{pmatrix} = -J^{-1} \begin{pmatrix} P_1 \\ \dots \\ P_n \end{pmatrix}. \tag{21}$$

Appendix D: Homotopy Continuation

Consider a system of n polynomial equations P_i in n unknowns X_j, denoted by $\mathbf{P}(\mathbf{X}) = \mathbf{0}$, with $\mathbf{P} = (P_1, \dots, P_n)^T$ and $\mathbf{X} = (X_1, \dots, X_n)^T$. To solve this system, we use the homotopy continuation method [30], itself a simple form of curve tracing. The principle of the method is as follows. Let $\mathbf{Q}(\mathbf{X}) = \mathbf{0}$ be another system of polynomial equations with the same total degree as $\mathbf{P}(\mathbf{X}) = \mathbf{0}$, but known solutions. A homotopy, parameterized by $t \in [0, 1]$, can be defined between the two systems by:

$$(1 - t)\mathbf{Q}(\mathbf{X}) + t\mathbf{P}(\mathbf{X}) = \mathbf{0}. \tag{22}$$

The solutions of the target system are found by tracing the curve defined in $\mathbb{R}^{n+1}$ by these equations from $t = 0$ to $t = 1$ according to step 1.4 of our curve tracing algorithm. In this case, however, the sample points are the known solutions of $\mathbf{Q}(\mathbf{X}) = \mathbf{0}$ at $t = 0$, which allows us to bypass step 1.3 of the algorithm. It can also be shown [30] that with an appropriate choice of $\mathbf{Q}$, the curve has no extrema or singularities, which allows us to also bypass steps 1.1-1.2.

References

1. V.I. Arnol'd. Singularities of systems of rays. *Russian Math. Surveys*, 38(2):87–176, 1983.
2. D.S. Arnon. Topologically reliable display of algebraic curves. *Computer Graphics*, 17(3):219–227, July 1983.
3. D.S. Arnon, G. Collins, and S. McCallum. Cylindrical algebraic decomposition I and II. *SIAM J. Comput.*, 13(4):865–889, November 1984.
4. C.L. Bajaj, C.M. Hoffmann, R.E. Lynch, and J.E.H. Hopcroft. Tracing surface intersections. *Computer Aided Geometric Design*, 5:285–307, 1988.
5. J. Callahan and R. Weiss. A model for describing surface shape. In *Proc. IEEE Conf. Comp. Vision Patt. Recog.*, pages 240–245, San Francisco, CA, June 1985.
6. G. Castore. Solid modeling, aspect graphs, and robot vision. In Pickett and Boyse, editors, *Solid modeling by computer*, pages 277–292. Plenum Press, NY, 1984.
7. I. Chakravarty. The use of characteristic views as a basis for recognition of three-dimensional objects. Image Processing Laboratory IPL-TR-034, Rensselaer Polytechnic Institute, October 1982.
8. S. Chen and H. Freeman. On the characteristic views of quadric-surfaced solids. In *IEEE Workshop on Directions in Automated CAD-Based Vision*, pages 34–43, June 1991.
9. G.E. Collins. *Quantifier Elimination for Real Closed Fields by Cylindrical Algebraic Decomposition*, volume 33 of *Lecture Notes in Computer Science*. Springer-Verlag, New York, 1975.
10. M.P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, Englewood Cliffs, NJ, 1976.
11. D. Eggert and K. Bowyer. Computing the orthographic projection aspect graph of solids of revolution. In *Proc. IEEE Workshop on Interpretation of 3D Scenes*, pages 102–108, Austin, TX, November 1989.
12. D. Eggert and K. Bowyer. Perspective projection aspect graphs of solids of revolution: An implementation. In *IEEE Workshop on Directions in Automated CAD-Based Vision*, pages 44–53, June 1991.
13. R.T. Farouki. The characterization of parametric surface sections. *Comp. Vis. Graph. Im. Proc.*, 33:209–236, 1986.
14. O.D. Faugeras and M. Hebert. The representation, recognition, and locating of 3-D objects. *International Journal of Robotics Research*, 5(3):27–52, Fall 1986.
15. Z. Gigus, J. Canny, and R. Seidel. Efficiently computing and representing aspect graphs of polyhedral objects. *IEEE Trans. Patt. Anal. Mach. Intell.*, 13(6), June 1991.
16. Z. Gigus and J. Malik. Computing the aspect graph for line drawings of polyhedral objects. *IEEE Trans. Patt. Anal. Mach. Intell.*, 12(2):113–122, February 1990.
17. W.E.L. Grimson and T. Lozano-Pérez. Localizing overlapping parts by searching the interpretation tree. *IEEE Trans. Patt. Anal. Mach. Intell.*, 9(4):469–482, 1987.
18. M. Hebert and T. Kanade. The 3D profile method for object recognition. In *Proc. IEEE Conf. Comp. Vision Patt. Recog.*, pages 458–463, San Francisco, CA, June 1985.
19. D.P. Huttenlocher and S. Ullman. Object recognition using alignment. In *Proc. Int. Conf. Comp. Vision*, pages 102–111, London, U.K., June 1987.
20. K. Ikeuchi and T. Kanade. Automatic generation of object recognition programs. *Proceedings of the IEEE*, 76(8):1016–35, August 1988.
21. J.T. Kajiya. Ray tracing parametric patches. *Computer Graphics*, 16:245–254, July 1982.
22. Y.L. Kergosien. La famille des projections orthogonales d'une surface et ses singularités. *C.R. Acad. Sc. Paris*, 292:929–932, 1981.
23. Y.L. Kergosien. Generic sign systems in medical imaging. *IEEE Computer Graphics and Applications*, 11(5):46–65, 1991.
24. J.J. Koenderink. *Solid Shape*. MIT Press, Cambridge, MA, 1990.
25. J.J. Koenderink and A.J. Van Doorn. The internal representation of solid shape with respect to vision. *Biological Cybernetics*, 32:211–216, 1979.

26. D.J. Kriegman and J. Ponce. Computing exact aspect graphs of curved objects: solids of revolution. *Int. J. of Comp. Vision.*, 5(2):119–135, 1990.
27. D.J. Kriegman and J. Ponce. On recognizing and positioning curved 3D objects from image contours. *IEEE Trans. Patt. Anal. Mach. Intell.*, 12(12):1127–1137, December 1990.
28. D.J. Kriegman and J. Ponce. Geometric modelling for computer vision. In *SPIE Conference on Curves and Surfaces in Computer Vision and Graphics II*, Boston, MA, November 1991.
29. D.J. Kriegman and J. Ponce. A new curve tracing algorithm and some applications. In P.J. Laurent, A. Le Méhauté, and L.L. Schumaker, editors, *Curves and Surfaces*, pages 267–270. Academic Press, New York, 1991.
30. A.P. Morgan. *Solving Polynomial Systems using Continuation for Engineering and Scientific Problems.* Prentice Hall, Englewood Cliffs, NJ, 1987.
31. H. Plantinga and C. Dyer. Visibility, occlusion, and the aspect graph. *Int. J. of Comp. Vision.*, 5(2):137–160, 1990.
32. J. Ponce and D.J. Kriegman. Computing exact aspect graphs of curved objects: parametric patches. In *Proc. AAAI Nat. Conf. Artif. Intell.*, pages 1074–1079, Boston, MA, July 1990.
33. J.H. Rieger. On the classification of views of piecewise-smooth objects. *Image and Vision Computing*, 5:91–97, 1987.
34. J.H. Rieger. The geometry of view space of opaque objects bounded by smooth surfaces. *Artificial Intelligence*, 44(1-2):1–40, July 1990.
35. J.H. Rieger. Global bifurcations sets and stable projections of non-singular algebraic surfaces. *Int. J. of Comp. Vision.*, 1991. To appear.
36. W.B. Seales and C.R. Dyer. Constrained viewpoint from occluding contour. In *IEEE Workshop on Directions in Automated "CAD-Based" Vision*, pages 54–63, Maui, Hawaii, June 1991.
37. T. Sripradisvarakul and R. Jain. Generating aspect graphs of curved objects. In *Proc. IEEE Workshop on Interpretation of 3D Scenes*, pages 109–115, Austin, TX, December 1989.
38. J. Stewman and K.W. Bowyer. Aspect graphs for planar-face convex objects. In *Proc. IEEE Workshop on Computer Vision*, pages 123–130, Miami, FL, 1987.
39. J. Stewman and K.W. Bowyer. Creating the perspective projection aspect graph of polyhedral objects. In *Proc. Int. Conf. Comp. Vision*, pages 495–500, Tampa, FL, 1988.
40. C.T.C. Wall. Geometric properties of generic differentiable manifolds. In A. Dold and B. Eckmann, editors, *Geometry and Topology*, pages 707–774, Rio de Janeiro, 1976. Springer-Verlag.
41. R. Wang and H. Freeman. Object recognition based on characteristic views. In *International Conference on Pattern Recognition*, pages 8–12, Atlantic City, NJ, June 1990.
42. N. Watts. Calculating the principal views of a polyhedron. CS Tech. Report 234, Rochester University, 1987.
43. C.E. Weatherburn. *Differential geometry.* Cambridge University Press, 1927.
44. T. Whitted. An improved illumination model for shaded display. *Comm. of the ACM*, 23(6):343–349, June 1980.